

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	CSA1205	Course Title:	Computer Architecture for Multicore Systems
C0 Assessed:	C05	Assessment:	ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)
Weightage:	30%	Total Marks:	25
C05 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	K. MRUDHULA	Reg.No.:	192524335
Department:	B. Tech AI&DS	Date:	31/8/26

ANNEXURE A

Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.

Algorithm DMA_Based_Data_Transfer

START

Input:

Source_Device

Destination_Memory

Data_Size

Output:

Data transferred successfully to memory

Step 1: Initialize DMA Controller

Step 2: Check whether DMA Controller is available

IF DMA Controller is NOT available THEN

```
    Display "DMA Controller Not Available"
    STOP
ENDIF
```

Step 3: Configure DMA Registers

```
    Source_Address ← Source_Device
    Destination_Address ← Destination_Memory
    Transfer_Count ← Data_Size
```

Step 4: Enable DMA Channel

Step 5: Send DMA Transfer Request

Step 6: CPU resumes execution of other processes

```
WHILE Transfer_Count > 0 DO
```

```
    DMA reads one block of data from Source_Device
```

```
    DMA writes the block into Destination_Memory
```

```
    Transfer_Count ← Transfer_Count - 1
```

```
END WHILE
```

Step 7: DMA generates an interrupt after transfer completion

```
IF Interrupt Received THEN
```

```
    Execute Interrupt Service Routine (ISR)
```

```
    Check Transfer Status
```

```
    IF Status = SUCCESS THEN
```

```
        Display "DMA Transfer Completed Successfully"
```

```
    ELSE
```

```
        Display "DMA Transfer Failed"
```

```
    ENDIF
```

```
ENDIF
```

Release DMA Channel

Display "CPU Continues Normal Execution"

STOP

Explanation

- The CPU initializes the DMA controller and configures the source address, destination address, and transfer size.
 - DMA independently transfers data between the I/O device and memory without continuous CPU intervention.
 - During the transfer, the CPU executes other processes, improving system efficiency.
 - After completing the transfer, DMA generates an interrupt.
 - The CPU executes the Interrupt Service Routine (ISR), verifies the transfer status, releases the DMA channel, and resumes normal execution.
-

2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.

Algorithm Vectored_Interrupt_Handler

START

Input:

Interrupt Requests from Multiple Devices

Output:

Appropriate Interrupt Service Routine Executed

Step 1: Initialize Interrupt Controller

Step 2: Load Interrupt Vector Table

Step 3: Assign Priority to Each Device

WHILE System is Running DO

IF Interrupt Request Received THEN

 Read Interrupt Vector Number

 Identify Interrupt Source

 IF Priority = HIGH THEN

```

        Execute High_Priority_ISR

    ELSE IF Priority = MEDIUM THEN
        Execute Medium_Priority_ISR

    ELSE
        Execute Low_Priority_ISR

    ENDIF

    Clear Interrupt Flag

    Return Control to CPU

ELSE

    Continue Normal Program Execution

ENDIF

END WHILE

Display "Interrupt Handling Completed"

STOP

```

Explanation

- The interrupt controller continuously monitors interrupt requests.
 - The interrupt vector table identifies the correct service routine.
 - High-priority devices are serviced first.
 - After servicing, the interrupt flag is cleared.
 - CPU resumes execution of the interrupted program.
-

3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.

Algorithm Dynamic_IO_Selection

START

Input:

Required_Latency

Required_Bandwidth

Output:

Selected I/O Communication Method

Step 1: Read Latency Requirement

Step 2: Read Bandwidth Requirement

IF Required_Bandwidth = HIGH OR Required_Latency = LOW THEN

IO_Method $\leftarrow$ Memory_Mapped_IO

ELSE

IO_Method $\leftarrow$ IO_Mapped_IO

ENDIF

Initialize Selected I/O Method

IF IO_Method = Memory_Mapped_IO THEN

Access Device through Memory Address

ELSE

Access Device through I/O Port

ENDIF

Verify Data Transfer

IF Transfer Successful THEN

Display "Communication Successful"

ELSE

Display "Communication Error"

ENDIF

STOP

Explanation

- The algorithm first checks bandwidth and latency requirements.
 - Memory-mapped I/O is selected for high-speed communication.
 - I/O-mapped I/O is selected for lower-speed devices.
 - The selected method is initialized, and communication status is verified before completion.
-

4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.

Algorithm Bus_Arbitration

START

Input:

Requests from PCIe, USB and Thunderbolt Devices

Output:

Bus Access Granted Successfully

Step 1: Initialize Bus Controller

Step 2: Create Empty Priority Queue

FOR Each Device Request DO

 Read Interface Type

 IF Interface = PCIe THEN

 Priority $\leftarrow$ 1

 ELSE IF Interface = Thunderbolt THEN

 Priority $\leftarrow$ 2

 ELSE

 Priority $\leftarrow$ 3

 ENDIF

 Insert Device into Priority Queue

END FOR

WHILE Queue is NOT Empty DO

 Select Highest Priority Device

 Grant Bus Access

 Perform Data Transfer

 Release Bus

END WHILE

Display "All Requests Completed Successfully"

STOP

Explanation

- The bus controller receives requests from multiple devices.
- Devices are assigned priorities based on their interface.
- PCIe receives the highest priority, followed by Thunderbolt and USB.
- Bus access is granted sequentially until all requests are completed.

-
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Algorithm Hybrid_IO_Communication

START

Input:

 Device_Type

 Data_Size

Output:

 Successful Data Communication

FOR Each Connected Device DO

IF Device_Type = LOW_SPEED THEN

Display "Using Polling Method"

WHILE Device is NOT Ready DO

 Check Device Status

END WHILE

Read Data

Store Data into Memory

ELSE

Display "Using DMA Transfer"

Initialize DMA Controller

Configure Source Address

Configure Destination Address

Start DMA Transfer

WAIT until Interrupt Received

Execute Interrupt Service Routine

Verify Transfer Status

ENDIF

END FOR

Display "All Devices Communicated Successfully"

STOP

Explanation

- Low-speed devices use polling because they require minimal bandwidth.
- High-speed devices use DMA for efficient data transfer.
- After DMA completes, an interrupt notifies the CPU.
- The CPU verifies the transfer and resumes normal execution.

- The hybrid approach improves overall system performance and CPU utilization.
-

Code of Conduct:

I K. MRUDHULA certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MRUDHULA K

MARKS ALLOCATION

	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach

Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient